CS F342 Comp Arch — Midsem Cheatsheet (2021–2025 papers)

Paper	Question → item #
21-22	1a→5, 1b→2, 1c→3, 1d→4, 1e→29, 2a→19, 2b→30, 2c→18, 3→31
22-23	1a→1, 1b→2, 1c→4, 1d→18, 2a→23, 2b→17, 2c→24, 2d→25, 2e→26, 3→36, 4→27, 5→33, 6→28
23-24	1a→6, 1b→9, 1c→7, 1d→18, 1e→5, 1f→15, 2a→20, 2b→22, 3a→32, 3b→19, 4→39, 5a→17, 5b→34
24-25	1a→8, 1b→9, 1c→14, 1d→4, 1e→17, 1f→18, 1g→16, 2→37, 3a→21, 3b→32, 3c→38
25-26	1a→12, 1b→13, 1c→35, 2→36, 3a→9, 3b→4, 3c→10, 3d→11, 4a→14, 4b→18, 4c→16

Tag after each heading = papers where it appeared (yr-Q). Same Q again in exam → write the same answer. Opcodes: R 000000, lw 100011, sw 101011, beq 000100, addi 001000, j 000010, jal 000011. Multicycle cycles: lw 5, sw 4, R 4, addi 4, beq 3, j 3.

A. THEORY

#1 Why opcode + funct fields instead of only opcode? *22-1a*

A 6-bit opcode can encode only 2^6=64 instructions. All R-type instrs share opcode 000000 and are told apart by the 6-bit **funct** → 64 more ALU ops without widening the 32-bit instruction.
R-type needs no 16-bit immediate, so its low bits are free for shamt+funct; I/J types need those bits for imm/address, so they get unique opcodes. Keeps all instrs 32-bit with fields at fixed positions (regularity → simple hardware).
Decoding becomes two-level: main decoder looks only at opcode (6 inputs); ALU decoder uses funct+ALUOp → smaller, faster control (see #10).

#2 Why is single-cycle control called a "Truth Table"? *21-1b, 22-1b*

In single-cycle MIPS every instr finishes in one clock, so the control unit is **purely combinational**: no state/memory, its outputs (RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, ALUOp, Jump) depend *only* on the present inputs (opcode, funct). A combinational circuit is completely specified by listing outputs for every input combination = a truth table (one row per opcode), which is then implemented as SOP logic/PLA/ROM. (Multicycle control is an FSM → state table, not truth table.)

#3 How does the processor capture the side effects when instrs complete? *21-1c*

Side effects = changes to architectural state: **register file**, **data memory** and **PC**. Combinational logic computes the new values during the cycle; they are captured only at the **rising clock edge** by state elements whose write enable is asserted: RF written if RegWrite=1 (WD3→reg A3), memory written if MemWrite=1 (WD→M[A]), PC←PC' every edge (multicycle: PCEn). Intermediate results in multicycle/pipeline are held in non-architectural registers (IR, MDR, A, B, ALUOut / pipeline regs) and committed in the write-back step.

#4 <<2 behaves as <<1 — which instrs affected? *21-1d, 22-1c, 24-1d, 25-3b*

Shift-by-2 converts a *word* offset to a *byte* offset; used only for PC-relative/jump targets.
beq/bne (taken): PCBranch = PC+4 + (SignImm<<1) = PC+4+2·imm instead of 4·imm → wrong, half-word aligned address (e.g., imm=3 → PC+4+6 not PC+4+12) → fetches garbage/misaligned instr. Not-taken branch still OK (PC+4).
j (and jal): JTA = {PC+4[31:28], addr<<1} → 26b<<1 = 27 bits only; target ≈ half the intended address, wrong/misaligned → jumps to wrong location.
Same in multicycle (ALUOut←PC+SignImm<<2 in Decode; jump path).
Not affected: R-type (add, sub, and, or, slt), lw, sw, addi — they don't use the shifters (lw/sw offsets are already byte offsets).

#5 Stuck control signals (single-cycle) *21-1a, 23-1e*

Signal	Malfunction & why
MemtoReg≡1	**R-type (add,sub,and,or,slt) & addi**: write ReadData (=M[ALUResult]) to reg instead of ALUResult. lw OK; sw, beq, j OK (RegWrite=0).
MemtoReg stuck-at-0	**lw only**: writes ALUResult (the address RF[rs]+imm) into rt instead of loaded data. All others OK.
ALUOp0≡1	ALUOp 00→01 ⇒ ALU **subtracts**: **lw, sw** (address = rs−imm), **addi** (rs−imm) wrong. beq (01) OK. R-type: 10→11, ALU decoder looks at funct when ALUOp1=1 (11 treated as 1X) → OK (if 11 were undefined, R-type would also fail). j OK.
RegDst≡0	**R-type**: result written to rt (20:16) instead of rd (15:11) → wrong reg & corrupts rt. lw/addi OK (use rt); sw, beq, j OK (no write).

#6 Can rt be the dest for R-type (and rd for others)? *23-1a*

Not advisable / no. MIPS reads registers from fixed fields: Read reg1 = rs (25:21), Read reg2 = rt (20:16) for *all* instrs, so the RF can be read **in parallel with decoding**. sw and beq must read rt as a source; if R-type wrote rt, its 2nd source would be in rd, so a mux (controlled by decoder) would be needed on **Read register 2** → RF read starts only after control decode → longer critical path (+t_{ctrl}+t_{mux}). Vice-versa (I-type writing rd) is impossible: bits 15:11 are part of the 16-bit immediate in lw/addi. Hence dest mux (RegDst) is on the write side: rt for I-type, rd for R-type.

#7 Do we need opcode in a special-purpose processor? *23-1c*

No. A single-purpose processor executes one fixed algorithm; the algorithm is hard-coded in the controller FSM which directly sequences the datapath control signals each state. There is no program/instruction memory, no fetch and no decode, so no opcode. Only status inputs (e.g., comparator outputs) feed the controller. Opcode is needed only in general-purpose (programmable) processors.

#8 Single-cycle vs multi-cycle control unit *24-1a*

Single-cycle	Multi-cycle
Combinational (truth table): signals = f(opcode,funct)	Sequential FSM: state reg + next-state logic + output logic; signals = f(state) (Moore), next state = f(state, opcode)
Signals constant for whole (long) cycle; CPI=1	Signals change every state; CPI 3–5
Signals: RegDst, RegWrite, ALUSrc, Branch, MemWrite, MemtoReg, ALUOp, Jump	Adds IorD, IRWrite, PCWrite, ALUSrcA, ALUSrcB[1:0], PCSrc[1:0]; enables for PC/IR
Separate I/D memories & adders → no sequencing	Shared memory & ALU reused across states
Only hardwired	Hardwired FSM or microprogrammed (#9)
Control table: Jump, RegDst, RegWrite, ALUSrc, Branch, ALUOp, MemRead, MemWrite, MemtoReg (R3)	States T0 IF, T1 ID, T2 EXE(lw/sw/addi), T3 MEM(lw), T4 WB(lw), T5 MEM(sw), T6 EXE(R), T7 WB(R), T8 EXE(beq), T9 WB(addi), T10 J (R7)

#9 How are states generated: hardwired vs microprogrammed? *23-1b, 24-1b, 25-3a*

Hardwired: current state kept in a **state register** (flip-flops, e.g., 4 bits for T0–T10 / S0–S11). **Next-state logic** (gates/PLA) computes S' = f(S, opcode, Zero) and loads it at each clock edge; **output logic** decodes S to control signals. (Alternative: sequence counter + decoder giving timing states T0,T1,…, cleared at end of instr.)
Microprogrammed: each state = one **microinstruction** stored in a control ROM; the state register is the **μPC** (address). Next state is generated by an address-select logic chosen by the μinstr's sequencing field: (i) μPC+1 (sequential), (ii) **dispatch ROM** indexed by opcode (Decode/MemAdr branching), (iii) jump to 0 (fetch). Control signals are read directly from the μinstr bits. Easy to modify, slower; hardwired faster.

#10 Why two-level control (main decoder + ALU decoder)? *25-3c*

Main decoder: opcode(6) → RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, **ALUOp[1:0]**. ALU decoder: ALUOp(2)+funct(6) → ALUControl. Benefits: (1) **smaller logic**: a monolithic controller would have 12 inputs (2^{12}=4096 rows) vs 2^6+2^8; (2) faster & simpler to design/verify; (3) **modular**: adding an R-type instr only changes the ALU decoder; ALU decoder reused in multicycle & pipeline; (4) funct only matters for R-type, so it isn't routed into the main decoder.

#11 PC enable: single vs multi-cycle? *25-3d*

Yes. Single-cycle: PC has **no enable** — every instruction completes in 1 cycle, so PC←PC' at every clock edge. Multicycle: an instr takes 3–5 cycles and the PC (memory address, PC+4, branch target computations) must hold its value during intermediate states, so PC has an enable: **PCEn = PCWrite + Branch·Zero** (PCWrite in Fetch & Jump states; Branch·Zero in beq state).

#12 Why can't single-cycle use one memory for fetch + data? *25-1a*

A single-ported memory has one address port and one read/write data port → only **one access per cycle**. Single-cycle lw/sw need instruction fetch AND data read/write in the **same** cycle → structural hazard (address port conflict). **Fix:** separate **instruction memory and data memory** (Harvard; split I-cache/D-cache) — or a dual-ported memory; (multicycle instead reuses one memory in different cycles with the IorD mux).

#13 Illegal control-signal combination *25-1b*

e.g. **RegWrite=1 with MemWrite=1** (during sw): memory is written correctly but the RF also writes Result (ALUResult = address, or garbage ReadData) into register rt/rd (whatever RegDst selects) → the source register (rt) being stored gets **corrupted**. Others: Branch=1 & Jump=1 (two PC sources, Jump mux overrides → wrong PC); MemtoReg=1 with MemRead=0 (writes undefined data).

#14 Methods of generating control signals in pipeline *24-1c, 25-4a*

Problem (slides 30–31): 10 control lines are needed in later stages, but while instr i is in EX, instr i+1 is being decoded → if controls are not stored, i+1's controls overwrite i's (erroneous signals).

1. **Strategy-1 (slide 27): pass the instruction, decode in every stage**: the instruction (opcode/funct) travels in IF/ID, ID/EX, EX/MEM, MEM/WB and each stage has its own control unit generating only its signals. *Adv:* narrow stage regs (just instr bits), simple per-stage control. *Disadv:* 4 duplicated decoders (area), decode delay added inside each stage's critical path.
2. **Strategy-2 (slide 28, P&H): decode once in ID, pass control bits**: same control unit as single-cycle (truth table) generates all 10 lines in ID; stage registers are extended with fields EX/M/WB: ID/EX 9 bits (RegDst, ALUOp[1:0], ALUSrc, Branch, MemRead, MemWrite, RegWrite, MemtoReg), EX/MEM 5 bits, MEM/WB 2 bits; each stage consumes its group (EX: RegDst, ALUOp, ALUSrc; MEM: Branch, MemRead, MemWrite; WB: RegWrite, MemtoReg; Jump used in IF). *Adv:* one decoder, reuses single-cycle CU, decode delay off the later stages. *Disadv:* wider pipeline regs (more flip-flops/power).
3. **Microprogrammed/ROM**: flexible, easy to change; slower, big ROM. Hazard unit separately generates Stall, Flush, Forward signals.

#15 Can forwarded data be taken before the stage (pipeline) registers? (R-type→R-type) *23-1f*

No. I1 computes its result in EX in cycle c; I2 needs it in its EX in cycle c+1. In cycle c+1 the wire before EX/MEM carries I2's *own* ALU output (I1's value is now in EX/MEM) → wrong data / combinational loop. Taking it in cycle c would chain ALU → forward mux → ID path in one cycle (longer clock). The pipeline register holds I1's result stable for the whole next cycle, so forward from **EX/MEM** (ALUOutM) and **MEM/WB** (ResultW) registers: ForwardAE=10 if (rsE≠0 & rsE==WriteRegM & RegWriteM), 01 if from WB.

#16 Setup & hold time; derive clock constraints *24-1g, 25-4c*

Setup t_{setup}: min time D must be stable *before* the clock edge. **Hold t_{hold}**: min time D must stay stable *after* the edge. Path R1 → combinational logic → R2:
Setup: data launched at edge 0 arrives at R2 latest at $t_{pcq}+t_{pd}$; must arrive t_{setup} before next edge: $\mathbf{T_c \ge t_{pcq} + t_{pd} + t_{setup}}$ ⇒ $t_{pd} \le T_c - (t_{pcq}+t_{setup})$ (sequencing overhead).
Hold: new data launched at the same edge must not reach R2 before t_{hold}; earliest arrival $t_{ccq}+t_{cd}$: $\mathbf{t_{ccq} + t_{cd} \ge t_{hold}}$ (independent of T_c; fix by adding buffers). With clock skew: $T_c \ge t_{pcq}+t_{pd}+t_{setup}+t_{skew}$; $t_{ccq}+t_{cd} \ge t_{hold}+t_{skew}$. (pcq/pd = propagation = max; ccq/cd = contamination = min.)

#17 128 registers & instruction set ×4 — effect on fields *22-2b, 23-5a, 24-1e*

Reg fields: $\log_2 128$ = **7 bits** (was 5). 4× instrs → opcode **6→8 bits** (and funct 6→8 if R-types also ×4).
R-type: op 8 + rs 7 + rt 7 + rd 7 + shamt 5 + funct 8 = **42 bits** (+10). **I-type**: op 8 + rs 7 + rt 7 + imm 16 = **38 bits** (+6).
If instruction must stay 32-bit: I-type imm shrinks to 32−8−14 = **10 bits** (smaller constants/branch range); R-type has only 32−8−21 = **3 bits** left for shamt+funct → must drop shamt/shrink funct or use more opcodes. Effects: bigger/slower RF, bigger decoder, fewer bits for constants.

B. NUMERICALS / TRACING

#18 5-stage pipeline: regs written/read in the 5th cycle *21-2c, 22-1d, 23-1d, 24-1f, 25-4b*

Cycle 5: I1 in **WB**, I2 MEM, I3 EX, **I4 in ID**, I5 IF. RF is **written in 1st half** of cycle (by I1) and **read in 2nd half** (by I4, which reads its rs & rt fields). (Prof: RF written @posedge, stage registers @negedge ⇒ write completes before I4's read is latched.)

Program (I1 … I5)	Written	Read (by I4)
21: or R2,R4,R5 / sw R5,72(R0) / sub R0,R1,R2 / **lw R3,15(R1)** / addi	R2 (or)	R1 (base; R3 on RD2 unused)
22: or \$t2,\$t4,\$t5 / sub / sw / **addi \$t1,\$t1,50** / lw	\$t2	\$t1
23,24: or \$t2,\$t4,\$t5 / sw / addi / **lw \$t3,15(\$t1)** / sub	\$t2	\$t1 (\$t3 unused)
25: addi \$t1,\$t1,50 / lw / or / **sw \$t5,72(\$t0)** / sub	\$t1	\$t0, \$t5

```
     c1 c2 c3 c4  c5
I1   IF ID EX MEM WB  ← writes (1st half)
I2      IF ID EX  MEM
I3         IF ID  EX
I4            IF  ID  ← reads (2nd half)
I5                IF
```

#19 Cycles & CPI of loop program (single/multi-cycle) *21-2a, 23-3b*

R1=0, R2=10; loop adds 2 to R1 → beq evaluated at R1=0,2,4,6,8 (not taken) and 10 (taken) ⇒ beq ×6, loop body (3 addi + j) ×5.
Instr count = 2 addi + 6 beq + 15 addi + 5 j = **28**.
Multicycle (addi 4, beq 3, j 3): 17×4 + 6×3 + 5×3 = 68+18+15 = **101 cycles**; CPI = 101/28 = **3.61**.
Single-cycle: 28 cycles, CPI = **1** (but long clock).

#20 Single-cycle fetches 0x0800248C: values in ALU ctrl, muxes, ALU, adders *23-2a*

0000 10|00 0000 0000 0010 0100 1000 1100 → op=000010 = **j** (J-type), addr26 = 0x000248C. (Field view: rs=0, rt=0, rd=00100, shamt=10010, funct=001100, imm=0x248C.) Assume PC=0x00400000, \$0=0.
Control (j, prof's table): Jump=1, RegWrite=0, MemRead=0, MemWrite=0, RegDst/ALUSrc/Branch/MemtoReg = X, ALUOp=XX.

Unit	Inputs → Output
A1 (PC+4)	0x00400000, 4 → **0x00400004**
SignExt / <<2	0x248C → 0x0000248C → 0x00009230
A2 (branch)	0x00400004 + 0x00009230 → 0x00409234
RF	A1=rs=0, A2=rt=0 → RD1=0, RD2=0
M2 RegDst	in0 = 00000 (rt), in1 = 00100 (rd) → X (RegWrite=0)
M3 ALUSrc	in0 = RD2 = 0, in1 = 0x0000248C → X
ALU decoder	ALUOp=XX, funct=001100 → ALUControl = XXXX (if ALUOp=00: 0010 add)
ALU	SrcA=RD1=0, SrcB=M3 out → result unused (X); Zero=X
Data mem	MemWrite=0 → not written; RD unused
M5 MemtoReg	in0 ALUResult, in1 ReadData → X (not written)
M4 PCSrc	Branch·Zero=0 → in0 = **0x00400004**
M1 Jump	in0 = 0x00400004, in1 = {PC+4[31:28]=0000, 0x000248C, 00} = 0x00009230; Jump=1 → **PC' = 0x00009230**

#21 Multicycle fetches 0x1000248C — values per state (M0–M6 of given fig) *24-3a*

0001 00|00 000|0 0000|0010 0100 1000 1100 → op=000100 = **beq \$0,\$0,0x248C** (rs=rt=0, imm=0x248C; rd bits=00100). Assume PC=0x00400000. 3 states:

State	Signals	Values
S0 Fetch	IorD=0, IRWrite=1, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite=1, Jump=0	M0: Adr=PC=0x00400000; Mem RD=0x1000248C→IR. M4=PC, M5=4, ALUCtrl=0010 → ALU=0x00400004; M6→M1 → PC=0x00400004 (also ALUOut)
S1 Decode	ALUSrcA=0, ALUSrcB=11, ALUOp=00	RF A1=A2=0 → A=0, B=0; SignImm=0x0000248C, <<2=0x00009230; M4=0x00400004, M5=0x00009230 → ALU=0x00409234 → ALUOut. M2 in: 0/4 (unused)
S8 Branch	ALUSrcA=1, ALUSrcB=00, ALUOp=01, Branch=1, PCSrc=01	M4=A=0, M5=B=0, ALUCtrl=0110 → 0−0=0, **Zero=1** ⇒ PCEn=Branch·Zero=1; M6=ALUOut → M1(Jump=0) → **PC=0x00409234** (branch taken)

M3 (MemtoReg) unused: RegWrite=0 throughout.

#22 Latencies (Table-1) & minimum clock period *23-2b*

mem 250, RF 150, mux 25, ALU 200, adder 150, gate 5, clk-q 30, setup 20, sext 50, control 50.

Instr	Critical path	ps
R-type	$t_{pcq}+t_{mem}+t_{RF}+t_{mux}$(ALUSrc)$+t_{ALU}+t_{mux}$(MemtoReg)$+t_{setup}$ = 30+250+150+25+200+25+20	700
LW	30+250+max(RF 150, sext+mux 75)+200+250(Dmem)+25+20	925
SW	30+250+150+200+250 (Dmem write)	880
BEQ	30+250+150+25+200+5(AND)+25(M4)+25(M1)+20(PC setup) [adder path 30+250+50+150=480 not critical]	730

Control (30+250+50=330 ps) is ready before RF data (430) → not on critical path. $\mathbf{T_{c,min}}$ **= max = 925 ps (lw)** → f ≈ 1.08 GHz.

#23 Bytes M3=00100000, M2=01000000, M1=00001000, M0=00000001 → instr *22-2a*

MIPS big-endian (M0 = MSB): 0000 0001 0000 1000 0100 0000 0010 0000 = 0x01084020 → 000000|01000|01000|01000|00000|100000 ⇒ **R-type: add \$t0, \$t0, \$t0** (op 0, rs=rt=rd=8, shamt 0, funct 0x20).
(If little-endian, M3 = MSB: 0x20400801 = 001000|00010|00000|0000100000000001 ⇒ I-type addi \$zero, \$v0, 2049.)

#24 PC = 0x30000000: range reachable by beq *22-2c*

Target = PC+4 + SignExt(imm)·4, imm 16-bit ∈ $[-2^{15}, 2^{15}-1]$. PC+4 = 0x30000004.

Min = 0x30000004 − 2^{17} = 0x30000004 − 0x20000 = **0x2FFE0004**

Max = 0x30000004 + (2^{15}−1)·4 = 0x30000004 + 0x1FFFC = **0x30020000**
≈ ±128 KB (−32768 … +32767 instructions from PC+4), word aligned.

#25 Horizontal μprogram control-memory width *22-2d*

Horizontal: one bit per control line → control field = **11 bits**. Condition select for 8 status bits: $\log_2 8$ = **3 bits** (MUX select). Next-address for 128 words: $\log_2 128$ = **7 bits**.

Width = 11 + 3 + 7 = **21 bits**; control memory = 128 × 21 bits. (If status were also one-hot: 11+8+7 = 26.)

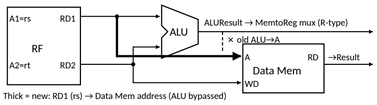
#26 Translate MIPS to C (D base \$s6, m=\$s0) *22-2e*

```
addi $t0,$s6,4    # t0 = &D[1]
add  $t1,$s6,$0   # t1 = &D[0]
sw   $t1,0($t0)   # D[1] = &D[0]
lw   $t0,0($t0)   # t0 = D[1] (= &D[0])
add  $s0,$t1,$t0  # m = &D[0] + D[1]
add  $s0,$s0,$t0  # m = m + D[1]
```

C: `D[1] = (int)&D[0]; m = (int)&D[0] + D[1] + D[1];` ⇒ m = 3·(base address of D).

#27 ld/sd (no offset) in single-cycle *22-4*

(a) Old lw: 30+250+150+200+250+25+20 = 925 ps. New ld needs no ALU: $t_{pcq}+t_{mem}+t_{RFread}+t_{mem}+t_{mux}+t_{RFwrite}$ = 30+250+150+250+25+20 = **725**. R-type = 30+250+150+25+200+25+20 = 700; beq ≈ 700. ⇒ $\mathbf{T_c}$ **= 725 ps** (ld still longest).
(b) Each lw/sw now needs an extra addi for the address: N' = N(1+0.26+0.12) = 1.38N. Old: N×925 = 925N. New: 1.38N×725 = 1000.5N ⇒ **slower**; 1000.5/925 = 1.082 → ≈ **8.2% slower**.
(c) The fraction f of loads/stores (that need an extra address instruction) vs the cycle-time gain: faster iff (1+f)·725 < 925 ⇒ **f < 27.6%**. (Here f = 38% → slower.)
(d) Data-memory address taken directly from RD1:

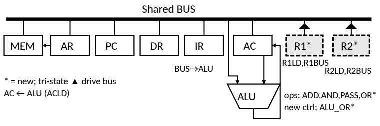


Thick = new: RD1 (rs) → Data Mem address (ALU bypassed)

C. DESIGN / DATAPATH MODIFICATIONS

#29 Simple CPU (shared bus): add OR R1,R2 (AC ← R1 ∨ R2) *21-1e*

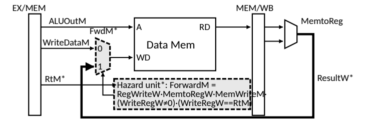
(1) Min components: registers **R1, R2** (if absent) with load (R1LD, R2LD) and **tri-state buffers** onto the bus (R1BUS, R2BUS); ALU gets an OR function (ALU_OR). Single bus ⇒ two operands can't travel together, so AC is used as temp: AC←R1, then AC←AC∨R2.



(2) Modified: ALU (OR op + ALU_OR input), control unit (decode new opcode; 7 instrs ⇒ 3-bit opcode, new states OR1, OR2, new signal equations), AC load logic, bus (R1, R2 drivers).
(3) States / μops: FETCH1: AR←PC; FETCH2: DR←M, PC←PC+1; FETCH3: IR←DR[op], AR←DR[addr]; **OR1: AC←R1** (R1→bus, ALU pass); **OR2: AC←AC∨R2** → FETCH1.
OR1 = I_{OR}·T3, OR2 = I_{OR}·T4 (I_{OR} = opcode decoder output). **R1BUS = OR1; R2BUS = OR2; ALU_OR = OR2; ALU_PASS = LD2 + OR1; ACLD = LD2 + ADD2 + AND2 + OR1 + OR2** (old terms + new); CLR seq. counter after OR2.

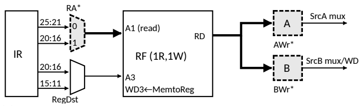
#30 Pipeline: hazard LW Ri,off(Rj) → SW Ri,off(Rk) *21-2b*

sw needs Ri only in MEM (as write data); lw's data is ready at end of its MEM ⇒ forward **MEM/WB → MEM** (no stall). Add RtM (5 b) to EX/MEM and a mux before Dmem WD.
Detect (ForwardM=1): RegWriteW · MemtoRegW · MemWriteM · (WriteRegW ≠ 0) · (WriteRegW == RtM). WriteDataM' = ForwardM ? ResultW : WriteDataM.
If base Rk == Ri (address needs the loaded value in EX) ⇒ load-use **stall** 1 cycle: lwstall = MemtoRegE · (RsD == RtE); modify old lwstall so sw data reg doesn't stall: lwstall = MemtoRegE·((RsD==RtE) + (RtD==RtE)·¬MemWriteD); StallF=StallD=FlushE=lwstall. R0 never forwarded.


EX/MEM
ALUOutM
FwdM*
WriteDataM
0
1
RtM*
A
RD
Data Mem
WD
MEM/WB
MemtoReg
ResultW*
Hazard unit*: ForwardM = RegWriteW·MemtoRegW·MemWriteM·(WriteRegW≠0)·(WriteRegW==RtM)

#31 Multicycle with 1-read-port RF *21-3*

(a) Add a **2:1 MUX, 5-bit** (inputs IR[25:21]=rs and IR[20:16]=rt, 1 select "RA", 5-bit output) on the single Read-register address. The single RD (32 b) feeds both A and B, so A and B get **write enables** (AWr, BWr). Modified: RF, A, B, controller (one extra state). **(b)**



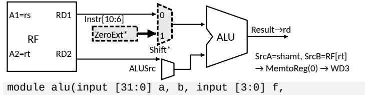
(c) FSM (unlisted signals = 0; RA: 0=rs, 1=rt). S-names = Harris; prof's T-names in R7 (S0=T0, S1=T1, S2=T2, S3=T3, S4=T4, S6=T6, S7=T7).

State	μop	Signals	Next
S0 Fetch	IR←M[PC]; PC←PC+4	IorD=0, IRWrite, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, PCWrite	S1
S1 Decode-1	A←RF[rs]; ALUOut←PC+SImm<<2	RA=0, AWr=1, ALUSrcA=0, ALUSrcB=11, ALUOp=00	R→S1b; lw→S2
S1b Decode-2	B←RF[rt]	RA=1, BWr=1	S6
S6 Execute	ALUOut←A op B	ALUSrcA=1, ALUSrcB=00, ALUOp=10	S7
S7 ALU WB	RF[rd]←ALUOut	RegDst=1, MemtoReg=0, RegWrite	S0
S2 MemAdr	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00 (sw: also RA=1, BWr)	S3
S3 MemRead	MDR←M[ALUOut]	IorD=1	S4
S4 Mem WB	RF[rt]←MDR	RegDst=0, MemtoReg=1, RegWrite	S0

Optimized: lw needs only rs ⇒ skips S1b (still 5 cycles); R-type 5 cycles (+1). sw reads rt during S2 (port free).

#32 Add sra \$rd,\$rt,shamt to single-cycle (+Verilog) *23-3a, 24-3b*

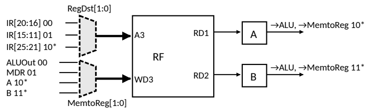
rd ← rt >>> shamt (arith). Shift amount = Instr[10:6] must reach ALU; add a **2:1 mux on SrcA** (RD1 / zero-ext shamt) with new signal **Shift**; ALU gets SRA op (ALUControl=1011): Result = \$signed(SrcB) >>> SrcA[4:0]. Main decoder: R-type row unchanged (RegWrite=1, RegDst=1, ALUSrc=0, MemtoReg=0, ALUOp=10). ALU decoder: ALUOp=10 & funct=000011 (table: 100011) → ALUControl=1011, Shift=1. Other instrs: Shift=0 → unaffected.



```
module alu(input [31:0] a, b, input [3:0] f,
           output reg [31:0] y, output zero);
 always @(*) case (f)
  4'b0000: y = a & b;
  4'b0001: y = a | b;
  4'b0010: y = a + b;
  4'b0110: y = a - b;
  4'b0111: y = ($signed(a)<$signed(b)) ? 1:0;
  4'b1011: y = $signed(b) >>> a[4:0]; // SRA
  default: y = 32'bx;
 endcase
 assign zero = (y == 32'b0);
endmodule
// new SrcA mux
assign SrcA = Shift ? {27'b0, Instr[10:6]}
                    : RD1;
// ALU decoder (with sra added)
always @(*) begin
 Shift = 0;
 case (ALUOp)
  2'b00: ALUControl = 4'b0010;   // add
  2'b01: ALUControl = 4'b0110;   // sub
  default: case (Funct)
   6'b100000: ALUControl = 4'b0010;
   6'b100010: ALUControl = 4'b0110;
   6'b100100: ALUControl = 4'b0000;
   6'b100101: ALUControl = 4'b0001;
   6'b101010: ALUControl = 4'b0111;
   6'b000011: begin                // sra
     ALUControl = 4'b1011; Shift = 1; end
   default:   ALUControl = 4'bxxxx;
  endcase
 endcase
end
```

#33 Add swap rs,rt to multicycle (Reg[rs]=Reg[rt]; Reg[rt]=Reg[rs]) *22-5*

One write port ⇒ two write-back states. **(a)** RegDst mux 2:1→**3:1** (add rs = IR[25:21]); MemtoReg mux 2:1→**4:1** (add A and B register outputs). RegDst, MemtoReg become 2-bit.



(b) Decode (S1) latches A=RF[rs], B=RF[rt]; op=SWAP → S12.

State	μop	RegDst	MemtoReg	RegWrite	Next
S12 Swap1	RF[rs]←B	10 (rs)	11 (B)	1	S13
S13 Swap2	RF[rt]←A	00 (rt)	10 (A)	1	S0

Other signals 0 (IorD, MemWrite, IRWrite, PCWrite, Branch = 0). A is re-latched at the same edge that writes RF[rs], so it still holds the *old* RF[rs] in S13 (IR unchanged). swap = 4 cycles.

#34 lwplus \$rt, imm(\$rs) in multicycle: RF[rt]←M[RF[rs]+imm]; RF[rs]←RF[rs]+4 *23-5b*

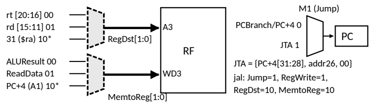
Yes, without changing the RF (1 write port): do the two writes in two different states. Datapath: RegDst mux 2:1→**3:1** adding IR[25:21] (rs) as input 10 (same as #33 left mux). ALU computes rs+4 using the existing "4" input of ALUSrcB while memory is read.

State	μop	Signals (rest 0)	Next
S0, S1	Fetch, Decode (std)	A←RF[rs]	S2
S2 MemAdr	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00	S3
S3 MemRd+Inc	MDR←M[ALUOut]; ALUOut←A+4	IorD=1, ALUSrcA=1, ALUSrcB=01, ALUOp=00	S4
S4 WB1	RF[rt]←MDR	RegDst=00, MemtoReg=1, RegWrite=1	S5'
S5' WB2	RF[rs]←ALUOut	RegDst=10, MemtoReg=0, RegWrite=1	S0

In S3 memory uses the old ALUOut (address) during the cycle; ALUOut is overwritten with A+4 only at the end edge. lwplus = 6 cycles.

#35 Add jal label to single-cycle *25-1c*

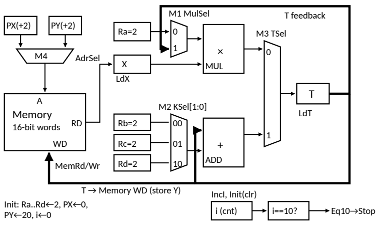
jal: PC ← JTA = {PC+4[31:28], addr26, 00} (same as j via Jump mux M1) **and** \$ra(\$31) ← PC+4. Modifications: **RegDst mux 2:1→3:1** (add constant 5'b11111 = 31); **MemtoReg mux 2:1→3:1** (add PC+4 from adder A1).



Main decoder row (op 000011): RegWrite=1, RegDst=10, ALUSrc=X, Branch=0, MemWrite=0, MemtoReg=10, ALUOp=XX, Jump=1. **New control:** RegDst and MemtoReg widened to 2 bits (equivalently one new signal **Link/JAL** driving two extra 2:1 muxes: WriteReg = JAL ? 31 : old; Result = JAL ? PC+4 : old). Existing instrs use 00/01 → unaffected. Return later with jr \$ra.

#36 16-bit multi-cycle single-purpose processor: $Y_i=aX_i^3+bX_i^2+cX_i+d$, a=b=c=d=2, 0≤i<10 *22-3, 25-2*

Horner: **Y = ((a·X + b)·X + c)·X + d** → 3 multiplies + 3 adds using 1 multiplier + 1 adder over several cycles. 16-bit data, byte-addressable ⇒ pointers step by **2**. $X_{0..9}$ at addresses 0,2,…,18; $Y_{0..9}$ at 20,…,38.



Control signals (13): Init, LdX, LdT, MulSel, KSel[1:0] (2), TSel, AdrSel, MemRd, MemWr, IncX, IncY, IncI. Status in: Eq10; output: Stop. Storage: Ra, Rb, Rc, Rd, X, T (16 b), PX, PY (16 b), i (4 b).

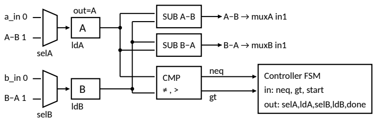
St	Operation	Asserted (rest 0)	Next
S0	Init: Ra..Rd←2, PX←0, PY←20, i←0	Init	S1
S1	if Eq10 → Stop; else X←M[PX], PX←PX+2	AdrSel=0, MemRd, LdX, IncX	Eq10? S9:S2
S2	T←a·X	MulSel=0, TSel=0, LdT	S3
S3	T←T+b	KSel=00, TSel=1, LdT	S4
S4	T←T·X	MulSel=1, TSel=0, LdT	S5
S5	T←T+c	KSel=01, TSel=1, LdT	S6
S6	T←T·X	MulSel=1, TSel=0, LdT	S7
S7	T←T+d (=Y_i)	KSel=10, TSel=1, LdT	S8
S8	M[PY]←T, PY←PY+2, i←i+1	AdrSel=1, MemWr, IncY, IncI	S1
S9	Done	Stop=1	S9

State reg 4 bits (10 states); total = 1 + 10×8 + 1 = 82 cycles. **Clock period (25-2c):** longest reg→reg path: $T_c = t_{clk\text{-}q} + \max(t_{mux}+t_{MUL}, t_{mux}+t_{ADD}, t_{mux}+t_{mem}) + t_{mux} + t_{setup}$ (+t_{ctrl} if select lines settle after data). MUL dominates: $\mathbf{T_c = t_{clk\text{-}q}+t_{mux}+t_{MUL}+t_{mux}+t_{setup}}$; e.g. 30+25+400+25+20 = 500 ps (assumed t_{MUL}=400). Memory path 30+25+250+20 = 325 ps.

#37 GCD in C, single-cycle datapath + controller, clock *24-2*

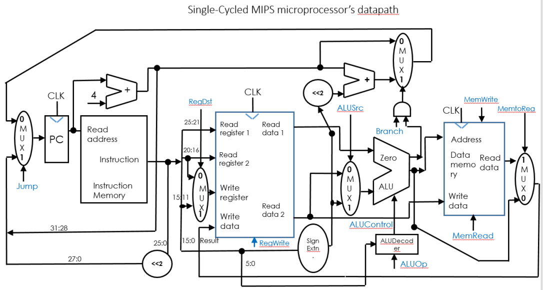
```
int GCD(int a, int b){
  while (a != b) {
    if (a > b) a = a - b;
    else       b = b - a;
  }
  return a;
}
```

(b) One loop iteration per clock. Datapath: regs A, B (ldA, ldB), 2:1 muxes (selA, selB) choosing input a/b or the difference, two subtractors (A−B, B−A), comparator (neq = A≠B, gt = A>B), out = A.

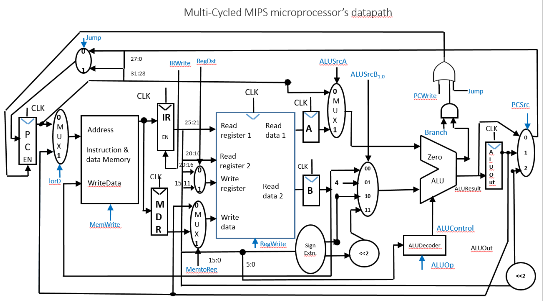


State	Cond	selA ldA selB ldB done	Next
S0 Load	—	0 1 0 1 0	S1
S1 Loop	neq=1, gt=1	1 1 X 0 0 (A←A−B)	S1
	neq=1, gt=0	X 0 1 1 0 (B←B−A)	S1
	neq=0	X 0 X 0 0	S2
S2 Done	—	X 0 X 0 1 (out=A)	S2/S0 on start

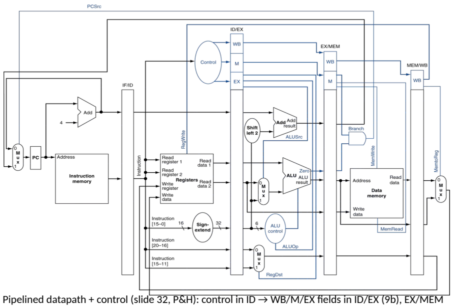
(c) Paths per cycle: data A,B → SUB → mux → reg; control A,B → CMP → FSM logic → mux select/load enable.
$\mathbf{T_c = t_{clk\text{-}q} + \max(t_{sub}, t_{cmp} + t_{ctrl}) + t_{mux} + t_{setup}}$. With Table-1 style values (clk-q 30, sub=cmp=adder 150, control 50, mux 25, setup 20): 30 + max(150, 200) + 25 + 20 = **275 ps**.



Single-cycle (exam fig): M1 Jump (left), M2 RegDst, M3 ALUSrc, M4 PCSrc (top), M5 MemtoReg; A1 = PC+4, A2 = branch adder.



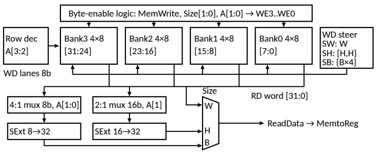
Multicycle (exam fig): M0 IorD, M1 Jump, M2 RegDst, M3 MemtoReg, M4 ALUSrcA, M5 ALUSrcB, M6 PCSrc; non-arch regs IR, MDR, A, B, ALUOut.



Pipelined datapath + control (slide 32, P&H): control in ID → WB/M/EX fields in ID/EX (9b), EX/MEM (5b), MEM/WB (2b).

#38 Memory size & memory unit for SW/SH/SB/LW/LH/LB *24-3c*

Size: 32-bit byte address ⇒ 2^{32} bytes = **4 GB** = 2^{30} words × 32 bits. Organise as **4 byte-wide banks** (2^{30}×8 each); A[1:0] picks byte lane, A[31:2] picks row. For 16 byte locations: 4 banks × 4 rows × 8 b, row = A[3:2].

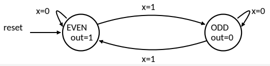


WE[3:0] (MemWrite=1): SW → 1111; SH → A[1]=0: 0011, A[1]=1: 1100; SB → one-hot of A[1:0] (2:4 decoder). Reads: LW = word; LH = SExt16(half sel. by A[1]); LB = SExt8(byte sel. by A[1:0]). Control from opcode: **Size[1:0]** (00 W, 01 H, 10 B), MemWrite, MemRead.

Components & sizes (full 32-bit): 4 banks of 2^{30}×8; row decoder 30→2^{30}; 2:4 decoder + gates for WE; write-steer 3:1 mux 32 b; read: 4:1 mux 8 b, 2:1 mux 16 b, sign-ext 8→32 & 16→32, 3:1 mux 32 b. (16-loc version: 4 banks 4×8, 2→4 row decoder.)

#39 Even number of 1s detector + testbench (Verilog) *23-4*

Moore FSM, 2 states: **EVEN** (out 1, reset/start state) and **ODD** (out 0); input 1 toggles state, input 0 keeps it. So output = 1 at the start (zero 1s = even).



```
module even_ones(input clk, rst, x,
                 output reg y);
  parameter EVEN = 1'b0, ODD = 1'b1;
  reg state, next;
  always @(posedge clk or posedge rst)
    if (rst) state <= EVEN;
    else     state <= next;
  always @(*) begin
    case (state)
      EVEN: next = x ? ODD  : EVEN;
      ODD : next = x ? EVEN : ODD;
    endcase
    y = (state == EVEN);   // 1 at start
  end
endmodule

module tb;
  reg clk = 0, rst = 1, x = 0;  wire y;
  even_ones dut(clk, rst, x, y);
  always #5 clk = ~clk;
  initial begin
    $monitor("%0t x=%b y=%b", $time, x, y);
    #12 rst = 0;
    x = 1; #10 x = 0; #10 x = 1; #10 x = 1;
    #10 x = 0; #10 x = 1; #10 $finish;
  end
endmodule
```

#28 NOPs, forwarding speed-up, code scheduling *22-6*

(a) No fwd: (n+0.4n)×250 = 350n ps. Fwd: (n+0.05n)×300 = 315n ps. Speed-up = 350/315 = **1.11**.

(b) Let x = NOP fraction without forwarding (forwarded version still 0.05n). Need (1+x)·250 > 1.05·300 = 315 ⇒ 1+x > 1.26 ⇒ **x > 26%** NOPs. (If NOPs scale proportionally, x/8 with fwd: 250(1+x) > 300(1+x/8) ⇒ x > 23.5%.)

(c) Assume RF written 1st half / read 2nd half ⇒ dependent instr must be ≥3 slots after producer (2 NOPs if adjacent).

(1) with NOPs

```
add $s3,$s1,$s0
nop
nop
lw  $s3,4($s3)
lw  $s1,0($s4)
nop
or  $s2,$s3,$s2
nop
nop
sw  $s2,0($s3) ;5 NOP
```

(2) best schedule

```
add $s3,$s1,$s0
lw  $s1,0($s4)
nop
lw  $s3,4($s3)
nop
nop
or  $s2,$s3,$s2
nop
nop
sw  $s2,0($s3) ;5 NOP
```

add→lw→or→sw is a true RAW chain (each needs 3 slots ⇒ sw at slot ≥10 ⇒ 10 slots − 5 instrs = **5 NOPs minimum**). Only lw $s1 is independent; it can't move above add (WAR: add reads $s1). So NOPs cannot go below 5 without forwarding (with forwarding: 0 NOPs needed).

D. REFERENCE — slides + Harris/P&H

R1 Instruction formats & 5 addressing modes

Type	31:26	25:21	20:16	15:11	10:6	5:0
R	op(6)=0	rs	rt	rd	shamt	funct
I	op	rs	rt	imm16 (offset)		
J	op	address26				

1. **Register**: all operands in regs — add $s0,$s1,$s2
2. **Immediate**: 16-bit constant in instr (sign-ext) — addi $s0,$s1,5
3. **Base (displacement)**: addr = RF[rs] + SignExt(imm) — lw $t0,8($s1)
4. **PC-relative**: target = PC+4 + SignExt(imm)·4 — beq/bne
5. **Pseudo-direct**: target = {PC+4[31:28], addr26, 00} — j, jal

Design principles: simplicity favours regularity; make the common case fast; smaller is faster; good design demands good compromises. MIPS good for pipelining (slide 34): all instrs 32-bit, few addressing modes & formats, memory operands only in lw/sw, operands aligned (x86 1–15 bytes → hard).

R2 Opcodes / funct (hex)

Instr	op	funct	Operation
add	000000	100000 (20)	rd=rs+rt
sub	000000	100010 (22)	rd=rs−rt
and / or	000000	100100 / 100101	rd=rs&rt / rs|rt
slt	000000	101010 (2A)	rd=(rs<rt)
sll / srl / sra	000000	000000 / 000010 / 000011	rd=rt<<,>>,>>> shamt
jr	000000	001000 (08)	PC=rs
lw / sw	100011 (23) / 101011 (2B)	—	rt=M[rs+imm] / M[rs+imm]=rt
beq / bne	000100 (04) / 000101 (05)	—	if(rs==/≠rt) PC=PC+4+imm·4
addi / slti	001000 (08) / 001010	—	rt=rs+SignExt(imm)
andi / ori / lui	001100 / 001101 / 001111	—	ZeroExt imm; lui rt=imm<<16
j / jal	000010 / 000011	—	PC=JTA; jal: $ra=PC+4

Regs: $0 zero | $1 $at | $2–3 $v0–1 | $4–7 $a0–3 | $8–15 $t0–7 | $16–23 $s0–7 | $24–25 $t8–9 | $26–27 $k0–1 | $28 $gp | $29 $sp | $30 $fp | $31 $ra.

R3 Main decoder truth table (single-cycle; prof slide 25)

Instr	op	Jmp	RDst	RWr	ASrc	Br	ALUOp	MRd	MWr	M2R
R-type	000000	0	1	1	0	0	10	0	0	0
lw	100011	0	0	1	1	0	00	1	0	1
sw	101011	0	x	0	1	0	00	0	1	x
addi	001000	0	0	1	1	0	00	0	0	0
beq	000100	0	x	0	0	1	01	0	0	x
j	000010	1	x	0	x	x	xx	0	0	x

(RDst=RegDst, RWr=RegWrite, ASrc=ALUSrc, Br=Branch, MRd/MWr=MemRead/Write, M2R=MemtoReg.) RegDst 0=rt,1=rd; ALUSrc 0=RD2,1=SignImm; MemtoReg 0=ALUResult,1=ReadData; PCSrc = Branch·Zero (bne: Branch·¬Zero); Jump selects JTA. (Pipelined table slide 29 shows sw MemRead=1 — use 0.)

Harris Verilog: `assign`
`{RegWrite,RegDst,ALUSrc,Branch,MemWrite,MemtoReg,Jump,ALUOp}=controls;` R 110000010, lw 101001000, sw 001010000, beq 000100001, addi 101000000, j 000000100.

R4 ALU decoder & ALU control lines / functions

ALUOp	funct	ALUCtrl 4b (prof/P&H)	3b (Harris)	action
00	x	0010	010	add (lw, sw, addi)
01	x	0110	110	sub (beq/bne)
10	100000	0010	010	add
10	100010	0110	110	sub
10	100100	0000	000	AND
10	100101	0001	001	OR
10	101010	0111	111	slt
10	000011	1011	—	sra (exam table)

4-bit ALU control lines = Ainvert | Bnegate | Operation[1:0] (00 AND, 01 OR, 10 add, 11 slt/less): 0000 AND, 0001 OR, 0010 add, 0110 sub (Bnegate+add, carry-in 1), 0111 slt, 1100 NOR. **Harris 3-bit F:** 000 A&B, 001 A|B, 010 A+B, 011 unused, 100 A&~B, 101 A|~B, 110 A−B, 111 SLT. F2 inverts B (B' = ~B, Cin=1 → subtract); SLT = sign bit of A−B, zero-extended. **Zero** = NOR of all result bits (used by beq).

R5 Single-cycle: Tc of every instruction (Harris/slide delays: t_{pcq} 30, t_{setup} 20, t_{mux} 25, t_{ALU} 200, t_{mem} 250, t_{RFread} 150, $t_{RFsetup}$ 20, $t_{RFwrite}$ 100)

Instr	Critical path	ps
lw	$t_{pcq}+t_{mem}+\max(t_{RFread}, t_{sext}+t_{mux})+t_{ALU}+t_{mem}+t_{mux}+t_{RFsetup}$ = 30+250+150+200+250+25+20	**925**
sw	$t_{pcq}+t_{mem}+t_{RFread}+t_{ALU}+t_{mem}$(write) = 30+250+150+200+250	880
R-type	$t_{pcq}+t_{mem}+t_{RFread}+t_{mux}+t_{ALU}+t_{mux}+t_{RFsetup}$ = 30+250+150+25+200+25+20	700
addi	$t_{pcq}+t_{mem}+t_{RFread}+t_{ALU}+t_{mux}+t_{RFsetup}$	675
beq	$t_{pcq}+t_{mem}+t_{RFread}+t_{mux}+t_{ALU}+t_{AND}+t_{mux}$(PCSrc)$+t_{mux}$(Jump)$+t_{setup}$	≈725+t_{AND}
j	$t_{pcq}+t_{mem}+t_{mux}+t_{setup}$ = 30+250+25+20	325

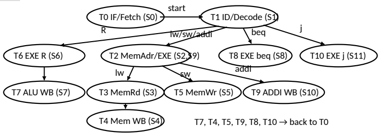
T_c = slowest = lw = 925 ps (slide/P&H version: 950 ps). Exec time (100×10^9 instrs) = #instr × CPI × T_c = 10^{11}×1×925 ps = **92.5 s**. Single-cycle needs separate IM/DM and 3 adders (PC+4, branch, ALU) — no FU reused in a cycle (non-shared FUs).

R6 Multicycle: Tc, CPI, non-architectural state

Architectural state (visible to programmer): PC, 32-reg RF, memory. **Non-architectural** (added in multicycle, 5 regs): **IR** (IRWrite), **MDR/Data**, **A**, **B**, **ALUOut** + extra muxes: IorD (shared I/D memory), ALUSrcA (PC/A), ALUSrcB 4:1 (B, 4, SignImm, SignImm<<2), PCSrc 3:1 (ALUResult, ALUOut, JTA). One memory, one ALU (no extra adders) — shared FUs.

$T_c = t_{pcq} + t_{mux} + \max(t_{ALU}+t_{mux}, t_{mem}) + t_{setup}$ = 30+25+250+20 = **325 ps**. Cycles: **lw 5, sw 4, R 4, addi 4, beq 3, j 3**. SPECINT mix 25% lw, 10% sw, 11% branch, 2% j, 52% R: CPI = .25·5+.10·4+.11·3+.02·3+.52·4 = **4.12**. Time = 10^{11}×4.12×325 ps = **133.9 s** (slower than single-cycle!). **Problems** (slide 4): lw split into 5 steps but T_c doesn't improve 5× (sequencing overhead per step); steps unequal; only one stage busy, others idle; 5 non-arch regs + extra muxes.

R7 Multicycle FSM — prof's T-states (slide 26) = Harris S-states



State	µop	Signals asserted (rest 0)
S0 Fetch	IR←M[PC]; PC←PC+4	IorD=0, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, IRWrite, PCWrite
S1 Decode	A←RF[rs], B←RF[rt]; ALUOut←PC+SImm·4	ALUSrcA=0, ALUSrcB=11, ALUOp=00
S2 MemAdr	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00
S3 MemRead	Data←M[ALUOut]	IorD=1
S4 MemWB	RF[rt]←Data	RegDst=0, MemtoReg=1, RegWrite
S5 MemWrite	M[ALUOut]←B	IorD=1, MemWrite
S6 Execute	ALUOut←A op B	ALUSrcA=1, ALUSrcB=00, ALUOp=10
S7 ALUWB	RF[rd]←ALUOut	RegDst=1, MemtoReg=0, RegWrite
S8 Branch	if(A==B) PC←ALUOut	ALUSrcA=1, ALUSrcB=00, ALUOp=01, PCSrc=01, Branch
S9 ADDIEx	ALUOut←A+SImm	ALUSrcA=1, ALUSrcB=10, ALUOp=00
S10 ADDIWB	RF[rt]←ALUOut	RegDst=0, MemtoReg=0, RegWrite
S11 Jump	PC←{PC[31:28],addr,00}	PCSrc=10, PCWrite

PCEn = PCWrite + Branch·Zero. Stage of each control (slide 26): Fetch: Jump; Execute: ALUSrc, ALUOp, RegDst; Memory: Branch, MemRead, MemWrite; WB: MemtoReg, RegWrite.

R8 Pipelined MIPS (slides 2–37)

Goal: CPI = 1 (from single-cycle) with $T_c < T_{multi}$ (from multicycle). Split into 5 stages: **IF, ID, EX, MEM, WB** with stage registers **IF/ID, ID/EX, EX/MEM, MEM/WB** (union of regs needed by all instrs). Partitioning rules: input of a sub-function comes totally from the previous one; no other interrelations; ≈equal delays. Each stage holds one element: IM 250 | RF 150 | ALU 200 | DM 250 (| RF write) ⇒ stage = slowest = **250 ps**.

Slide 13: single-cycle latency 950 ps, throughput 1/950 ps = 1.05×10^9 instr/s. Pipelined: latency 5×250 = 1250 ps, throughput 1/250 ps = 4×10^9 instr/s. Latency unchanged/longer, throughput ideally ×5 (= #stages). n instrs on k stages: (k + n − 1) cycles; speedup = $n \cdot T_{single}/((k+n-1) \cdot T_{stage}) \to k$ (ideal, balanced).

Control: 10 lines generated in ID like single-cycle, then carried in extended stage registers (else instr i+1 overwrites instr i's controls): ID/EX holds **9 bits** (EX: RegDst, ALUOp[1:0], ALUSrc; M: Branch, MemRead, MemWrite; WB: RegWrite, MemtoReg), EX/MEM **5 bits** (M+WB), MEM/WB **2 bits** (WB). Jump used in IF/ID. Strategy-1: pass the instruction through every stage register and decode a local control in each stage. Strategy-2 (P&H): decode once, pass WB/M/EX control fields. Prof: **RF write @posedge, stage regs write @negedge** (so write-then-read in same cycle).

Harris pipelined T_c (branch in ID, fwd): max{ $t_{pcq}+t_{mem}+t_{setup}$ (F); $2(t_{RFread}+t_{mux}+t_{eq}+t_{AND}+t_{mux}+t_{setup})$ (D); $t_{pcq}+t_{mux}+t_{mux}+t_{ALU}+t_{setup}$ (E); $t_{pcq}+t_{memwrite}+t_{setup}$ (M); $2(t_{pcq}+t_{mux}+t_{RFwrite})$ (W) } = 2(150+25+40+15+25+20) = **550 ps**; CPI≈1.15 → 10^{11}×1.15×550 ps = **63.3 s**.

	Single	Multi	Pipeline
T_c	925	325	550 (250 ideal)
CPI	1	4.12	1.15 (1 ideal)
Time	92.5 s	133.9 s	63.3 s
FUs	non-shared	shared	shared by several instrs

Hazards: structural (resource), data (RAW), control (branch). **Forwarding:** if (rsE≠0 & rsE==WriteRegM & RegWriteM) ForwardAE=10; else if (rsE≠0 & rsE==WriteRegW & RegWriteW) ForwardAE=01; else 00 (same for rtE→ForwardBE). **lw stall:** lwstall = MemtoRegE·((rsD==rtE)+(rtD==rtE)); StallF=StallD=FlushE=lwstall. **Branch in ID:** ForwardAD = (rsD≠0)(rsD==WriteRegM)RegWriteM; branchstall = BranchD·RegWriteE·(WriteRegE==rsD + WriteRegE==rtD) + BranchD·MemtoRegM·(WriteRegM==rsD + WriteRegM==rtD); FlushD = PCSrcD. Without forwarding: RAW needs 2 NOPs (write 1st half/read 2nd half); P&H branch resolved in MEM → 3-cycle penalty.

R9 IPC > 1: superscalar vs VLIW (slides 38–47)

CPI = cycles/instr; IPC = 1/CPI; time = #instr × (1/IPC) × T. **Static 2-issue (VLIW)**: 64-bit issue packet = ALU/branch + load/store; compiler pairs them, fills with **noop**, may remove all hazards. Adds: 32 more bits from IM, 2 more RF read ports + 1 write port, extra adder for address. **Dynamic (superscalar)**: hardware issues 2–3 instrs/cycle; in-order issue → reservation stations → out-of-order execute (FUs) → in-order commit; scoreboard, Tomasulo. Differences: general vs special instructions; dynamic vs static scheduling; HW vs compiler hazard detection; instruction format; VLIW needs recompilation for each VLIW machine.

Interconnect \ FUs	Less FUs	More FUs
Less (single bus)	multi-cycle, IPC<1	multi-cycle, IPC<1
More (many buses)	multi-cycle, IPC<1	single-cycle / pipeline, IPC=1

ISA/processor design steps (slide 48): 1 instructions for the algorithm; 2 choose strategy (shared bus / single / multi / pipeline) & design datapath per instr; 3 combine datapaths (insert muxes); 4 clock period from critical path (+setup, clk-to-Q; satisfy hold); 5 identify control signals; 6 design CU (hardwired or microprogrammed); 7 test & verify.

R10 Shared bus; hardwired vs microprogrammed

Shared bus: one common set of wires connecting all registers, ALU and memory. Every source drives the bus through a **tri-state buffer** (output enable, e.g. PCBUS, DRBUS); only **one** source may be enabled per cycle (else bus contention); destinations latch with LD signals. ⇒ one transfer per bus per cycle, so an instruction takes several cycles (multi-cycle, IPC<1) but needs little interconnect/hardware. Simple CPU fetch: FETCH1 AR←PC; FETCH2 DR←M, PC←PC+1; FETCH3 IR←DR[op], AR←DR[addr]; then execute states (e.g. LD: DR←M; AC←DR). Control by state decoder ($I_x \cdot T_i$) and OR-ing state terms for each signal.

	Hardwired	Microprogrammed
Implementation	FSM: state reg + gates/PLA (or seq. counter + decoder)	µPC + control ROM + µsequencer (address-select)
Speed	fast	slower (ROM access per state)
Change/add instr	redesign logic	rewrite microcode
Large/complex ISA	hard	easy (CISC, x86)
Used in	RISC (MIPS), pipelines	CISC

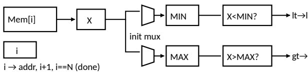
µinstruction = control field + condition-select + next-address. **Horizontal**: 1 bit per control line (wide, parallel, fast, no decode). **Vertical**: encoded fields (narrow, needs decoders, slower). Next µaddress: µPC+1, dispatch ROM (opcode), branch on condition, or 0 (fetch).

R11 Timing formulas

Setup: **$T_c \geq t_{pcq} + t_{pd} + t_{setup}$ ($+t_{skew}$)**; Hold: **$t_{ccq} + t_{cd} \geq t_{hold}$ ($+t_{skew}$)**. t_{pd} = max (propagation) delay, t_{cd} = min (contamination). $f_{max} = 1/T_c$. Sequencing overhead = $t_{pcq}+t_{setup}$. CPU time = #instr × CPI × T_c; IPC = 1/CPI; MIPS = $f/(CPI \cdot 10^6)$; speedup = T_{old}/T_{new}; $CPI_{avg} = \Sigma f_i \cdot CPI_i$. Dynamic discipline: inputs stable in aperture [t_{setup} before, t_{hold} after] edge.

R12 MinMax single-purpose processor (generic — prof's slides not provided)

Find MIN and MAX of N numbers in memory. Datapath: regs X, MIN, MAX, i (addr/counter); comparators **lt**=(X<MIN), **gt**=(X>MAX), **done**=(i==N); incrementer i+1; muxes to init MIN/MAX. Controls: ldX, ldMin, ldMax, selInit, clrI, incI.



State	Action	Next
S0	i←0; X←M[0]; MIN←X, MAX←X (selInit)	S1
S1	i←i+1; if done → S3; X←M[i]	S2
S2	if lt: MIN←X; if gt: MAX←X	S1
S3	Done, output MIN, MAX	S3

Pipelined MinMax: stage 1 load X[i] | stage 2 compare | stage 3 update MIN/MAX, one element per cycle. $T_c = t_{clk\text{-}q} + t_{mux} + t_{cmp} + t_{ctrl} + t_{setup}$.

R13 C++ simulation — single-cycle MIPS

```
#include <cstdint>
uint32_t PC=0, RF[32], IM[1024], DM[1024];
void step(){                         // one clock
 uint32_t in=IM[PC>>2];              // fetch
 uint32_t op=in>>26, rs=(in>>21)&31,
  rt=(in>>16)&31, rd=(in>>11)&31,
  sh=(in>>6)&31, fn=in&63;
 int32_t imm=(int16_t)(in&0xFFFF); // sign-ext
 uint32_t a=RF[rs], b=RF[rt];       // RF read
 uint32_t pc4=PC+4, next=pc4;
 switch(op){
 case 0x00: switch(fn){             // R-type
   case 0x20: RF[rd]=a+b; break;
   case 0x22: RF[rd]=a-b; break;
   case 0x24: RF[rd]=a&b; break;
   case 0x25: RF[rd]=a|b; break;
   case 0x2A: RF[rd]=(int)a<(int)b; break;
   case 0x03: RF[rd]=(int)b>>sh; break;
  } break;
 case 0x23: RF[rt]=DM[(a+imm)>>2]; break;//lw
 case 0x2B: DM[(a+imm)>>2]=b; break;   //sw
 case 0x08: RF[rt]=a+imm; break;        //addi
 case 0x04: if(a==b) next=pc4+(imm<<2);
            break;                      //beq
 case 0x02: next=(pc4&0xF0000000)|
            ((in&0x3FFFFFF)<<2); break; //j
 }
 RF[0]=0; PC=next;                  // clock edge
}
int main(){ /*load IM*/ long c=0;
 while(IM[PC>>2]){ step(); c++; } } // CPI=1
```

R14 C++ simulation — multicycle MIPS (FSM)

```
enum St{FETCH,DECODE,MEMADR,MEMRD,MEMWB,
 MEMWR,EXEC,ALUWB,BRANCH,ADDIEX,ADDIWB,JUMP};
uint32_t PC,IR,A,B,ALUOut,Data,RF[32],M[2048];
St s=FETCH; long cyc=0, ins=0;
void clk(){ cyc++;
 uint32_t op=IR>>26,rs=(IR>>21)&31,
  rt=(IR>>16)&31,rd=(IR>>11)&31,fn=IR&63;
 int32_t imm=(int16_t)IR;
 switch(s){
 case FETCH: IR=M[PC>>2]; PC+=4; ins++;
   s=DECODE; break;
 case DECODE: A=RF[rs]; B=RF[rt];
   ALUOut=PC+(imm<<2);           // br target
   s= op==0?EXEC : (op==0x23||op==0x2B)?MEMADR
    : op==4?BRANCH : op==8?ADDIEX : JUMP; break;
 case MEMADR: ALUOut=A+imm;
   s= op==0x23?MEMRD:MEMWR; break;
 case MEMRD: Data=M[ALUOut>>2]; s=MEMWB; break;
 case MEMWB: RF[rt]=Data; s=FETCH; break;
 case MEMWR: M[ALUOut>>2]=B; s=FETCH; break;
 case EXEC: ALUOut= fn==0x20?A+B: fn==0x22?A-B
   : fn==0x24?A&B: fn==0x25?A|B
   : (int)A<(int)B; s=ALUWB; break;
 case ALUWB: RF[rd]=ALUOut; s=FETCH; break;
 case BRANCH: if(A==B) PC=ALUOut; s=FETCH; break;
 case ADDIEX: ALUOut=A+imm; s=ADDIWB; break;
 case ADDIWB: RF[rt]=ALUOut; s=FETCH; break;
 case JUMP: PC=(PC&0xF0000000)|((IR&0x3FFFFFF)<<2);
   s=FETCH; break; }
 RF[0]=0; }       // CPI = cyc/ins
```

R15 Multicycle datapath in Verilog (Harris)

```
module datapath(input clk, reset, PCEn, IorD,
  IRWrite, RegDst, MemtoReg, RegWrite, ALUSrcA,
  input [1:0] ALUSrcB, PCSrc, input [2:0] ALUCtrl,
  input [31:0] ReadData, output [5:0] Op, Funct,
  output Zero, output [31:0] Adr, WriteData);
 wire [31:0] PC,PCNext,Instr,Data,RD1,RD2,A,SrcA,
   SrcB,ALUResult,ALUOut,SignImm,WD3; wire [4:0] WA3;
 flopenr #(32) pcreg(clk,reset,PCEn,PCNext,PC);
 mux2 #(32) adrmux(PC,ALUOut,IorD,Adr);
 flopenr #(32) ir(clk,reset,IRWrite,ReadData,Instr);
 flopr  #(32) mdr(clk,reset,ReadData,Data);
 mux2 #(5)  wamux(Instr[20:16],Instr[15:11],
                 RegDst,WA3);
 mux2 #(32) wdmux(ALUOut,Data,MemtoReg,WD3);
 regfile rf(clk,RegWrite,Instr[25:21],Instr[20:16],
            WA3,WD3,RD1,RD2);
 signext se(Instr[15:0],SignImm);
 flopr #(32) areg(clk,reset,RD1,A);
 flopr #(32) breg(clk,reset,RD2,WriteData);
 mux2 #(32) srcamux(PC,A,ALUSrcA,SrcA);
 mux4 #(32) srcbmux(WriteData,32'd4,SignImm,
           {SignImm[29:0],2'b00},ALUSrcB,SrcB);
 alu alu(SrcA,SrcB,ALUCtrl,ALUResult,Zero);
 flopr #(32) aluoutreg(clk,reset,ALUResult,ALUOut);
 mux3 #(32) pcmux(ALUResult,ALUOut,
   {PC[31:28],Instr[25:0],2'b00},PCSrc,PCNext);
 assign Op=Instr[31:26]; assign Funct=Instr[5:0];
endmodule
// controller: PCEn = PCWrite | (Branch & Zero);
module flopenr #(parameter W=8)(input clk,reset,en,
  input [W-1:0] d, output reg [W-1:0] q);
 always @(posedge clk, posedge reset)
  if (reset) q <= 0; else if (en) q <= d;
endmodule   // flopr = same without en
module regfile(input clk, we3, input [4:0] ra1,
  ra2, wa3, input [31:0] wd3, output [31:0] rd1,rd2);
 reg [31:0] rf[31:0];
 always @(posedge clk) if (we3) rf[wa3] <= wd3;
 assign rd1 = (ra1!=0) ? rf[ra1] : 0;
 assign rd2 = (ra2!=0) ? rf[ra2] : 0;
endmodule
```